COURSEWORK PROJECT · SOFTWARE ENGINEERING

Book-shop Automation Software (BAS)

Requirements analysis, UML design, and a working Node / React implementation for a single-shop point-of-sale and inventory system.

Group I
SE Lab Project

Agenda

01
Requirements & Stack
Scope, roles, tech choices

02
Data Flow
Level 0 & Level 1 DFDs

03
Use Case & Class
Actors and the object model

04
Sequence Diagrams
Enquiry, sale, reorder

05
State & Activity
Title lifecycle, sale flow

06
Data Model & API
Schema, pages, endpoints

07
Testing
Frontend, backend, integration

08
Wrap-up
Repo, build order, next steps

REQUIREMENTS & ROLES

What BAS automates, and for whom

Four functions

- Customer availability enquiry (search by title/author)
- Point-of-sale (scan ISBN, print receipt)
- Inventory receiving (new supply, price, rack)
- Owner reporting (demand, sales stats, reorder list)

Four roles

Role	Auth	Can do
Customer	No	Search, request a book
Sales Clerk	Yes	Record sales
Employee	Yes	Receive supply, price/rack
Owner	Yes	All reports, reorder list

TECHNOLOGY STACK

Node / React, chosen for a small transactional system

Backend

Node.js, Express, TypeScript
PostgreSQL + Prisma ORM
JWT auth, bcrypt hashing

Frontend

React, TypeScript, Vite
TailwindCSS, React Router
TanStack Query

Testing & tooling

Vitest, Supertest, RTL
Playwright (integration)
ESLint, Prettier, Docker Compose

`Invariant` Money is stored as integer paise everywhere — never floats. ISBN is the database primary key for Book.

HOW IT WORKS

Four services, two stores

A customer's search hits the **Catalogue** directly. A miss creates a stub request (R.2); an in-catalogue title with zero stock bumps its request counter (R.3) instead of failing silently.

A clerk's scan writes to both stores at once: **Catalogue** stock goes down, and the **Sales Log** gets a dated, priced line — the two facts a reorder computation and a revenue report each need.

Every report reads, never writes. The reorder list is the one exception that reads both stores together: stock from the Catalogue, 14-day sales from the Log.

D1

Catalogue

Books, stock, price, rack, requestCount

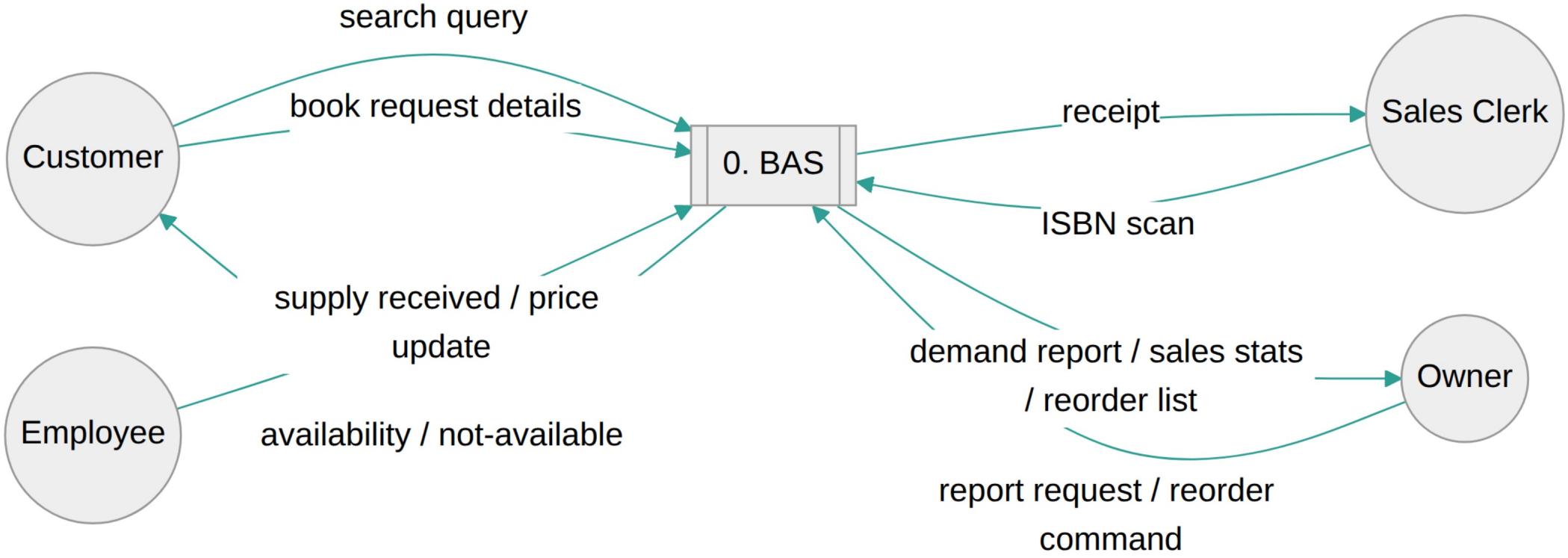
D2

Sales Log

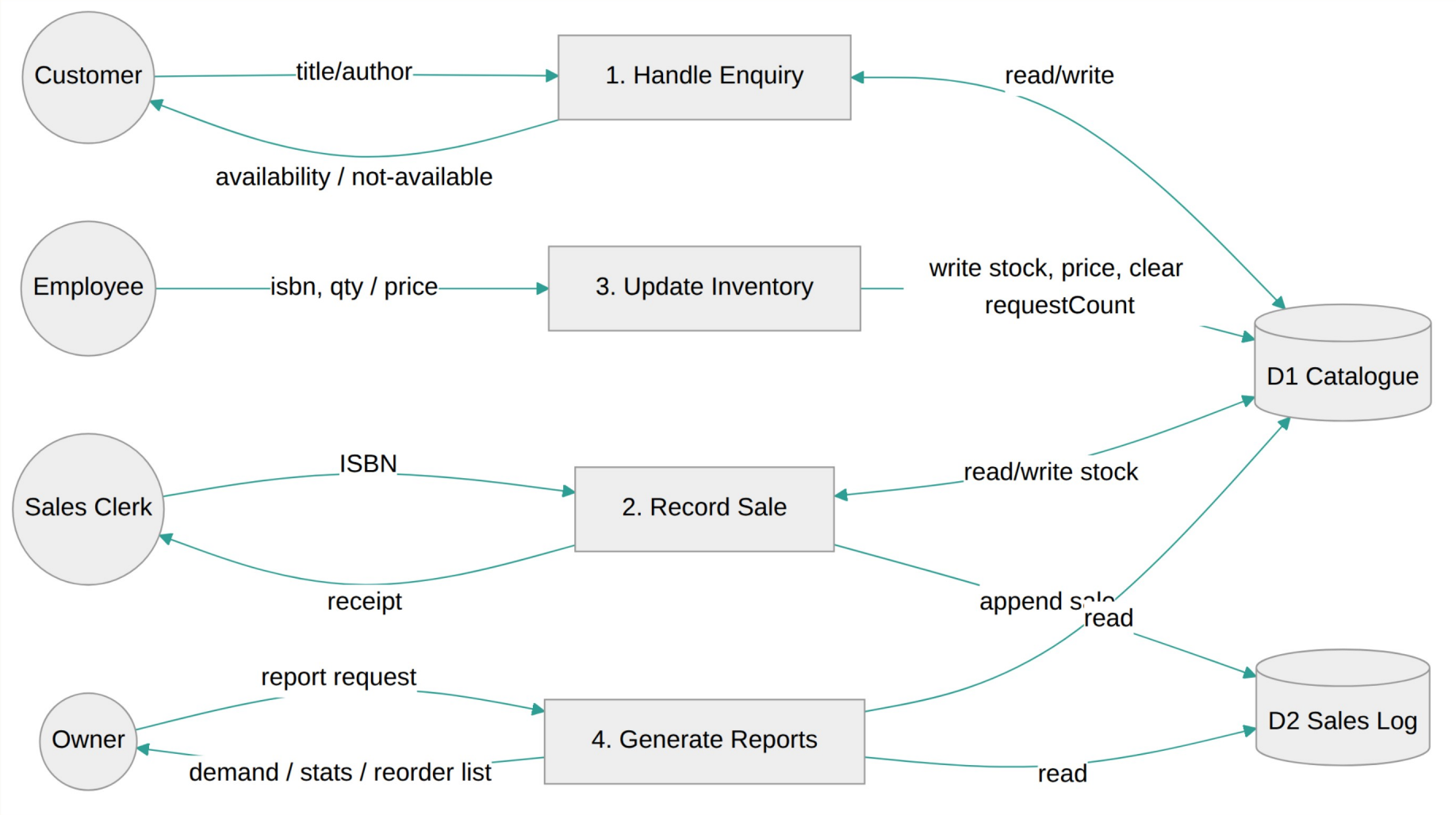
Append-only: isbn, date, unit price

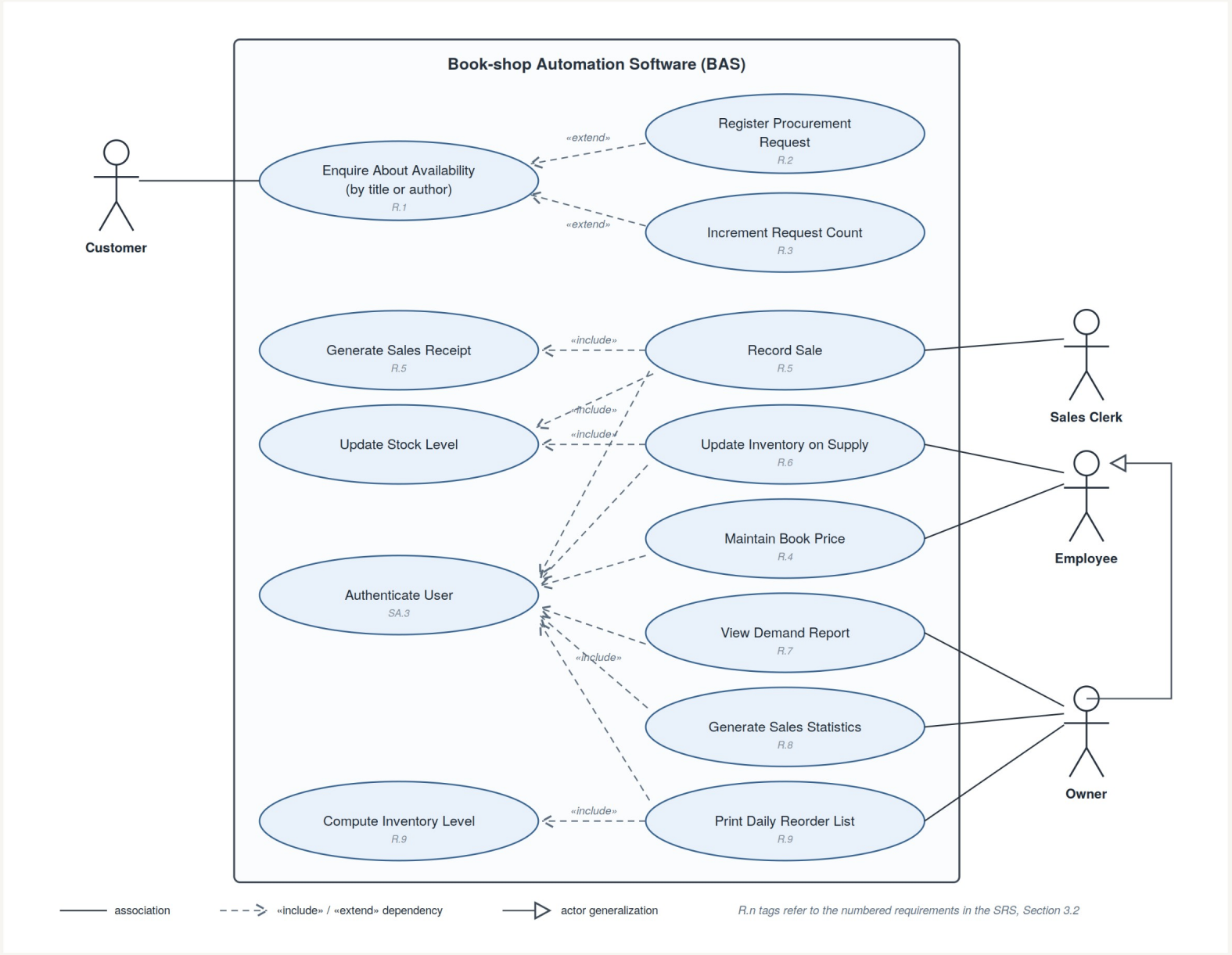
System context

Four external entities, one process

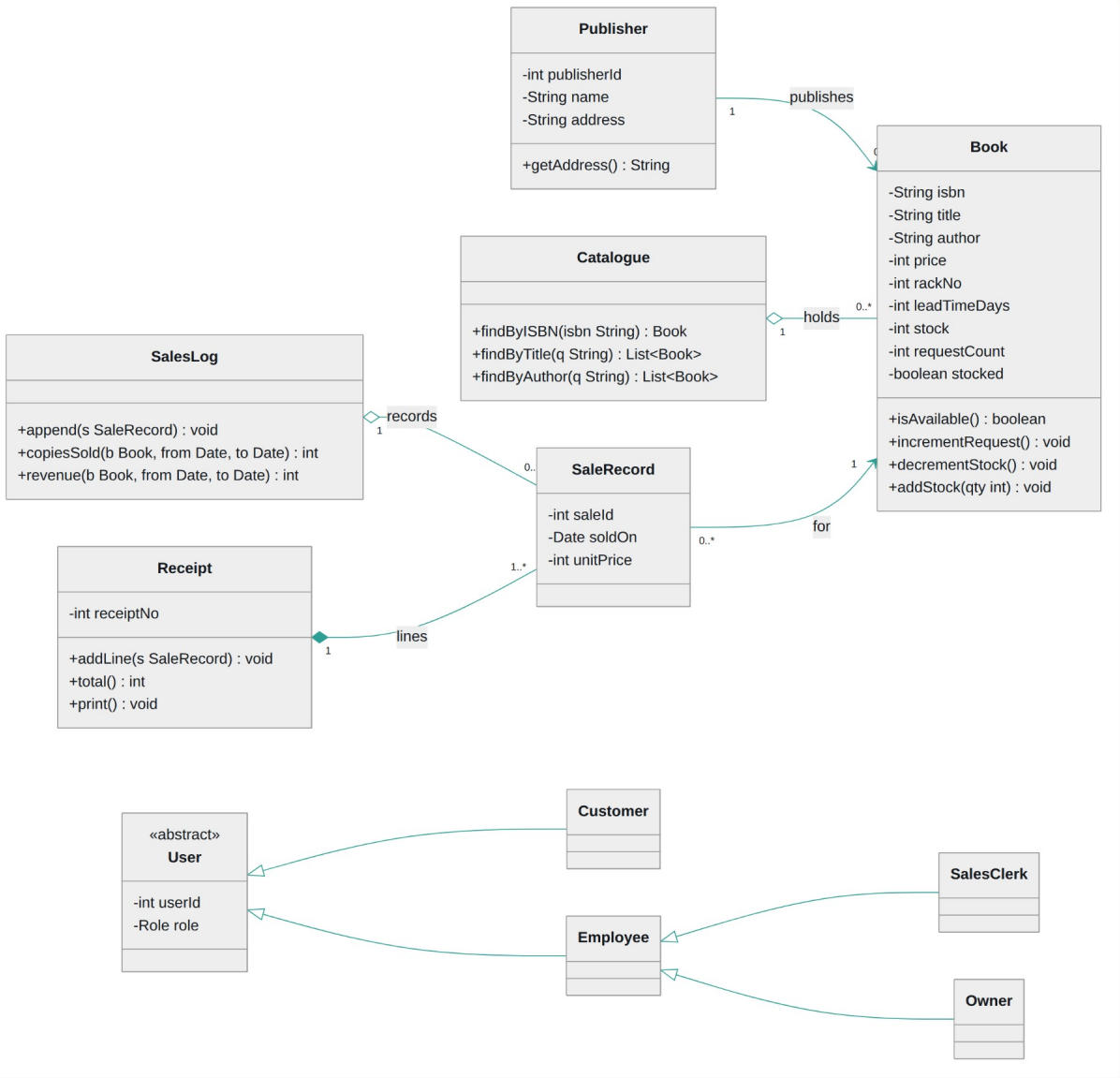


Four processes, two data stores

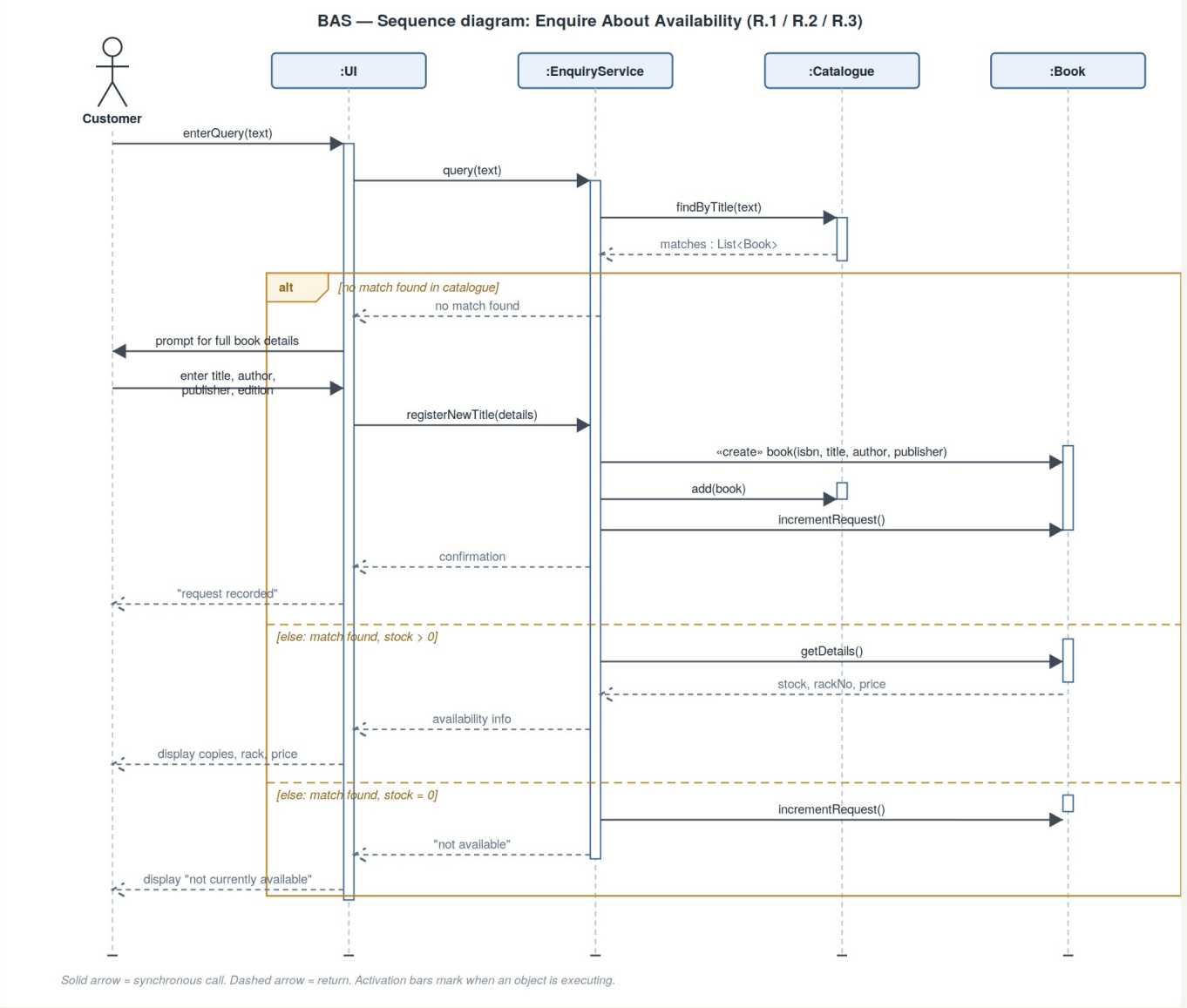



Book-shop Automation Software (BAS)
Customer
Enquire About Availability
(by title or author)
R.1
Register Procurement
Request
R.2
Increment Request Count
R.3
«extend»
«extend»
Generate Sales Receipt
R.5
Record Sale
R.5
Update Stock Level
Update Inventory on Supply
R.6
Maintain Book Price
R.4
Authenticate User
SA.3
View Demand Report
R.7
Generate Sales Statistics
R.8
Compute Inventory Level
R.9
Print Daily Reorder List
R.9
«include»
«include»
«include»
«include»
«include»
Sales Clerk
Employee
Owner
association
«include» / «extend» dependency
actor generalization
R.n tags refer to the numbered requirements in the SRS, Section 3.2

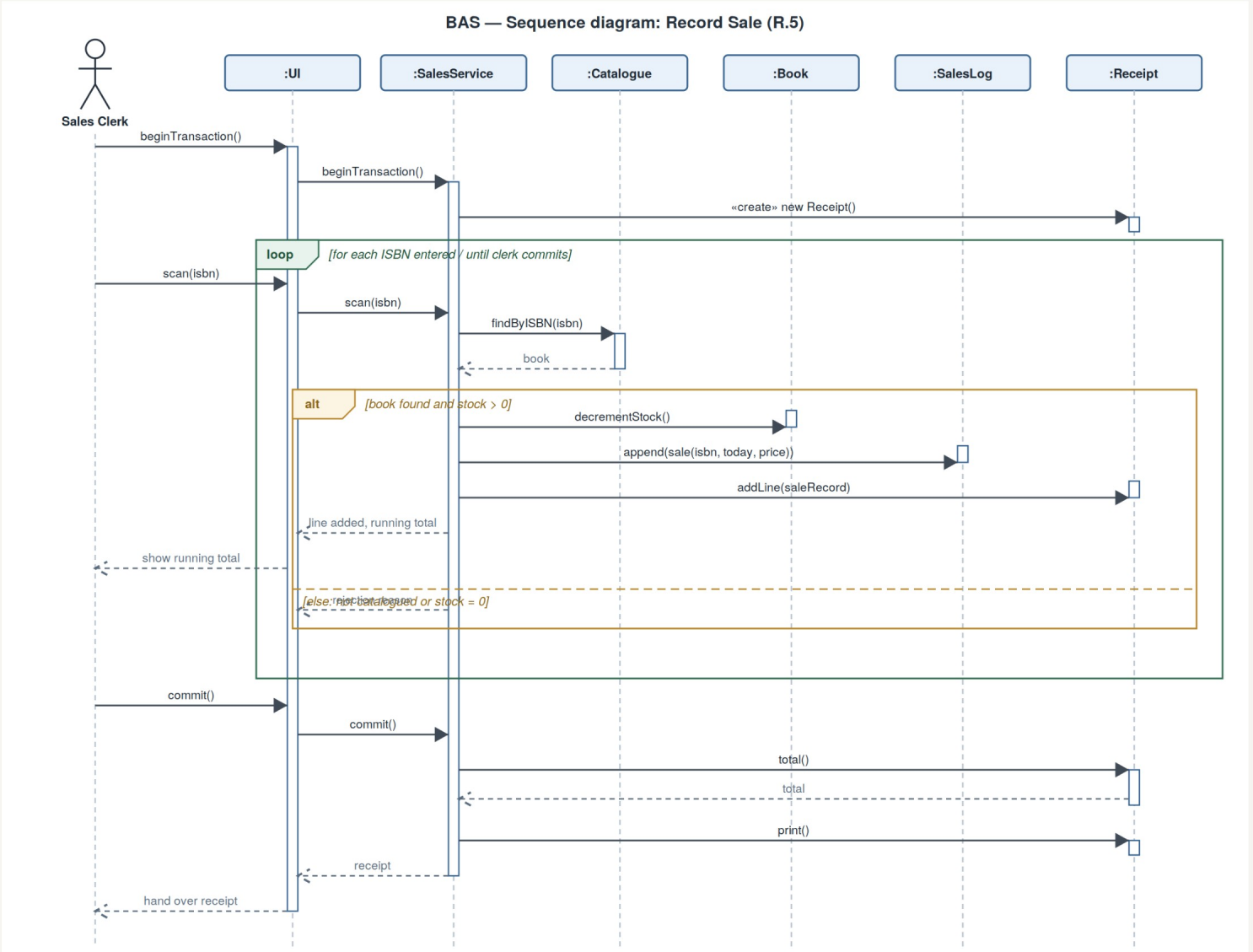
Receipt composes SaleRecord; SaleRecord keeps its own price



R.1 / R.2 / R.3 · one 3-way alt

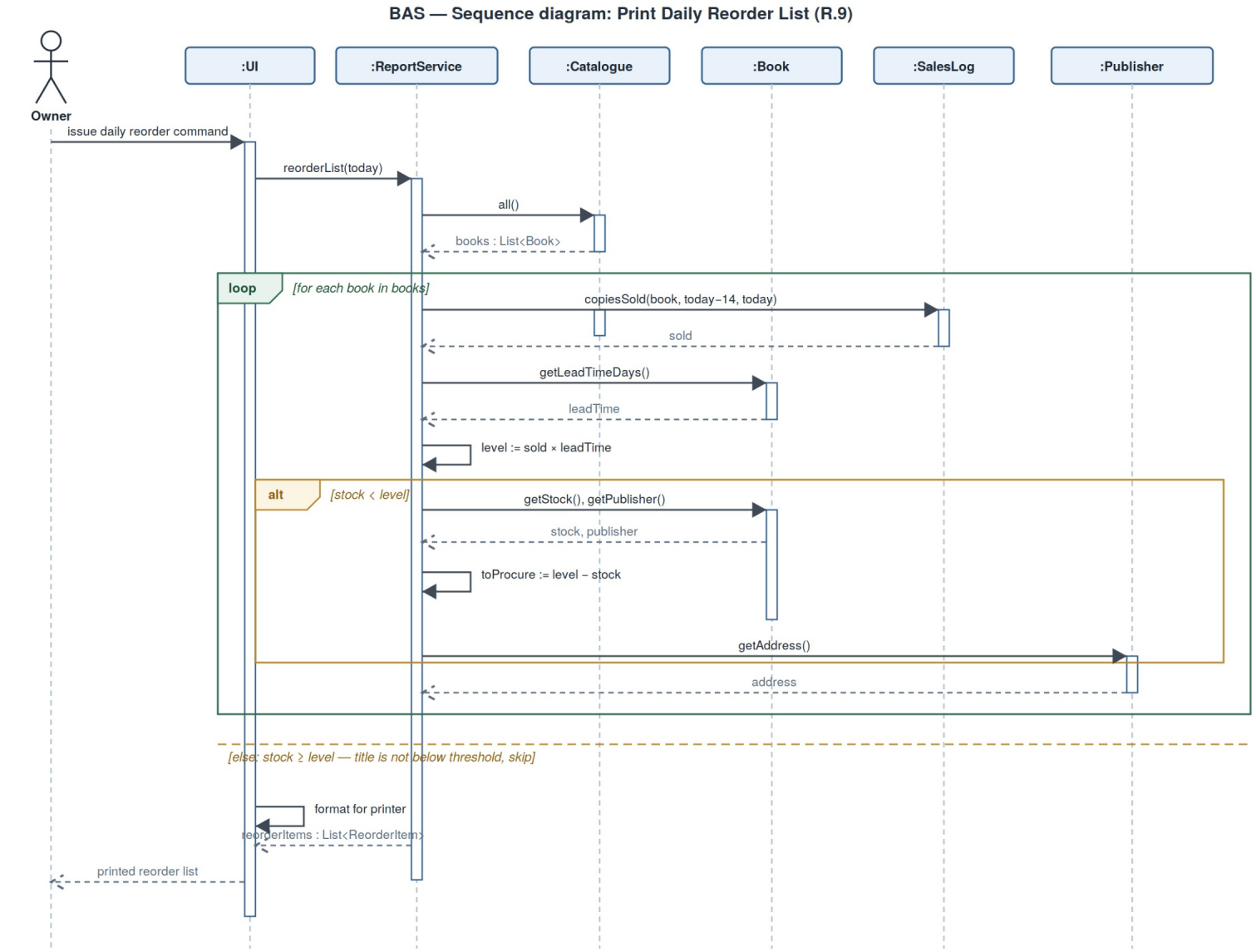


R.5 · loop with a nested alt for rejection



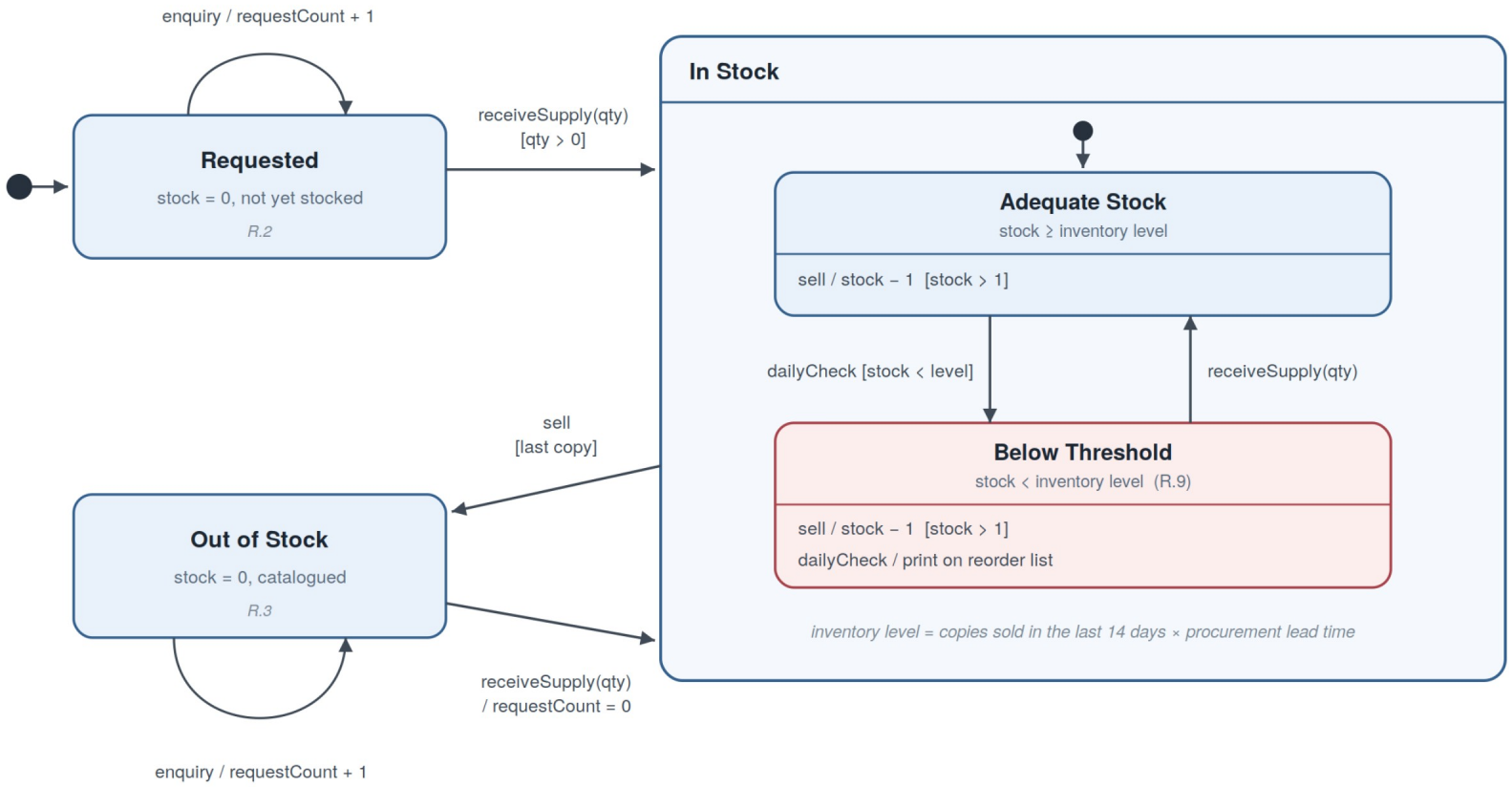
SRS A.6: one ISBN entry per copy sold. A rejected entry does not abort the transaction (SRS SA.1: atomic commit only at the end).

R.9 · level = sold(14d) × leadTime



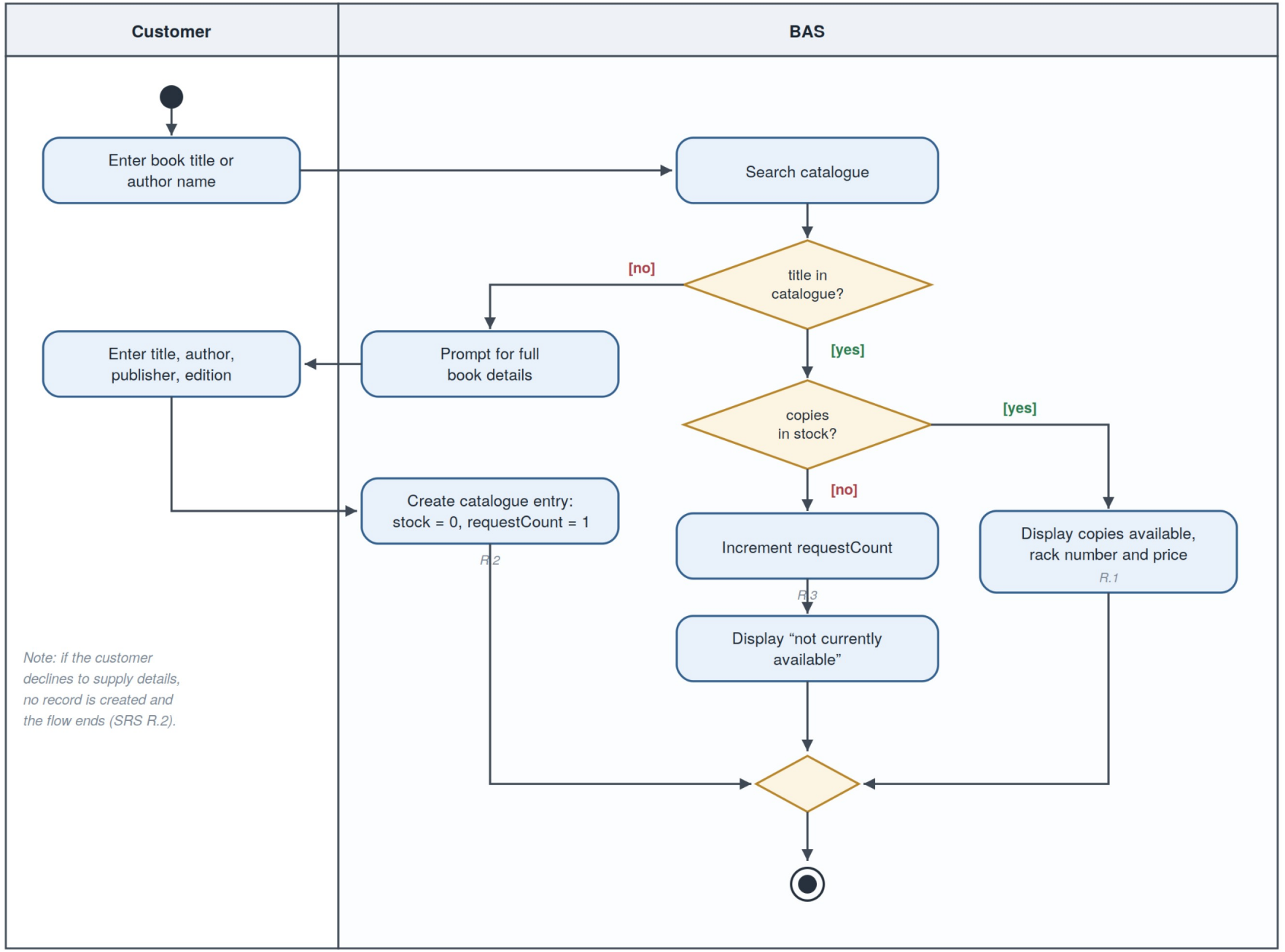
A title with no sales in the last fortnight has level = 0 and never satisfies stock < level, so it never appears in the reorderItems.

BAS — State diagram: lifecycle of a title record



Transition label format: event [guard] / action. Text below the divider inside a state is an internal transition: it fires without leaving the state. "In Stock" is a composite state; its two substates differ only by the R.9 threshold test.

BAS — Activity diagram: Enquire About Availability (R.1 / R.2 / R.3)



DATA MODEL

Four Prisma models

model
Publisher
id, name, address
→ has many Book

model
Book
isbn (PK), title, author
priceInPaise, rackNo
leadTimeDays, stock
requestCount, stocked

model
SaleRecord
id, isbn (FK)
soldOn, unitPriceInPaise
receiptNo

model
User
id, username
passwordHash
role: CLERK | EMPLOYEE | OWNER

Key point SaleRecord keeps its own unitPriceInPaise — a later price change never rewrites past revenue.

BUSINESS RULES

The rules that must match exactly

- inventoryLevel = copiesSold(last 14 days) × leadTimeDays
- belowThreshold = stock < inventoryLevel
- toProcure = max(0, inventoryLevel − stock)
- Zero sales in 14 days → level = 0, never on the reorder list
- Stock never goes below zero — rejected, not clamped
- Receiving supply resets requestCount to 0
- A sale transaction is atomic: all accepted lines commit together
- Price changes apply to future sales only

FRONTEND PAGES

Eight pages, each with one job

Page	Route	Access	Distinct from the others
Search	/	Public	Landing page, no auth needed
Book Detail	/books/:isbn	Public	Result of a click, not a search itself
Login	/login	Public	Only page customers never need
POS	/pos	Clerk	Only page that writes sales
Inventory	/inventory	Employee	Writes stock up; POS writes it down
Demand Report	/reports/demand	Owner	Read-only, no date range
Sales Statistics	/reports/sales	Owner	Read-only, has a date range
Reorder List	/reports/reorder	Owner	Computed daily, not queryable by range

REST API · BASE PATH /API

Nine endpoints

Method	Path	Auth	Notes
POST	/auth/login	None	Returns token + role
GET	/books?title=&author=	None	A stock=0 match increments requestCount (R.3)
GET	/books/:isbn	None	404 if unknown
POST	/books/requests	None	R.2 — creates a stub, stock=0
PATCH	/books/:isbn/price	Employee	400 if priceInPaise ≤ 0
POST	/books/:isbn/receive	Employee	Clears requestCount
POST	/sales	Clerk	Atomic; see next slide
GET	/reports/demand	Owner	requestCount > 0, sorted desc
GET	/reports/sales?from=&to=	Owner	400 if from > to
GET	/reports/reorder	Owner	Computed against server date

POST /SALES · WORKED EXAMPLE

Rejection doesn’t abort the transaction

Request

```
{
  "items": [
    "9780131103627",
    "9780131103627",
    "9780201896831"
  ]
}
```

Response · 201

```
{
  "receiptNo": 4821,
  "lines": [ 3 items with unitPriceInPaise ],
  "rejected": [ ],
  "totalInPaise": 210000
}
```

A rejected item (not catalogued, or stock = 0) appears in rejected with a reason — the rest of the transaction still commits.

TESTING · 1 OF 3

Frontend edge cases

- Empty search submitted
- Double-click submit on the request form
- Two browser tabs open on /pos for one clerk
- from-date after to-date on Sales Statistics
- SQL-injection-style search text treated as literal
- JWT expires mid-transaction on POS
- Malformed ISBN input
- Browser back button mid-transaction on POS

TESTING · 2 OF 3

Backend edge cases

- Sell the last copy, then race a second sale attempt
- Two employees call /receive on the same ISBN concurrently
- Duplicate /books/requests resolving to the same ISBN → 409
- priceInPaise or qty of zero/negative
- leadTimeDays = 0 → level always 0
- Reorder check inside the shop's first 14 days
- Sales stats query with zero sales in the period
- Customer-role token hitting /reports/* → 403

TESTING · 3 OF 3

Integration & frontend–backend fallacies

- Optimistic stock decrement rolled back on server rejection
- Stale cached book list after another clerk's concurrent sale
- Retried /sales commit must be idempotent (no double-decrement)
- Frontend running total vs backend authoritative total on commit
- "Today" for reorder/stats always computed server-side, never trusted from the client
- Network drop mid-scan — confirm no partial silent commit
- Receipt orphaned by a timeout — must be retrievable by receiptNo
- Price change mid-transaction must not affect the in-flight sale

Thank you

Group I SE Lab